



ICPC Asia West Continent Championship 2025-26

A - Ant and Sticks

You are given two distinct points on an infinite table, $a = (a_x, a_y)$ and $b = (b_x, b_y)$.

You also have m sticks, each of which is a line segment of length w . You may place the sticks anywhere on the table, with arbitrary position and orientation. The sticks may overlap. However, no stick may contain point a or point b .

An ant starts at a and wants to reach b . It may walk anywhere on the table, but it cannot cross any stick.

You want to place the sticks so that the length of the ant's shortest path from a to b is as large as possible.

If it is possible to block the ant completely, so that no path from a to b exists, then the answer is -1 .

Otherwise, let L be the supremum[†], over all valid stick placements, of the length of the shortest path from a to b .

If there exists a placement for which this value is attained, then L is simply the maximum. If it is not attained by any single placement, but can be approached arbitrarily closely, you must still output L .

[†]The supremum (least upper bound) of a set of real values is the smallest real number that is greater than or equal to every value in the set. It may or may not be attained by some value in the set.

Input Format

- The first line of input will contain a single integer t , denoting the number of test cases.
- The first and only line of each test case contains 6 space-separated integers a_x , a_y , b_x , b_y , m , and w .

Output Format

For each test case:

- print -1 if the ant can be blocked completely;
- otherwise print one real number — the value L defined above.

Let B be the correct answer and A be your output. Your output is accepted if and only if

$$|A - B| \leq 10^{-6} \cdot \max(1, |B|).$$

Constraints

- $1 \leq t \leq 10^4$
- $0 \leq a_x, a_y, b_x, b_y \leq 10^9$
- $1 \leq m, w \leq 10^9$
- It is guaranteed that $(a_x, a_y) \neq (b_x, b_y)$

Sample Input 1

```
2
0 0 1 1 343 434
4 5 786331192 707731892 1 639711202
```

Sample Output 1

```
-1
1425074478.231629371643066
```

Explanation

- **Test case 1:** It can be proven that it is possible to block the ant completely by placing the sticks optimally.

B - Based Family Constraints

There are n people standing in a line. For each position i , the i -th person belongs to family a_i . Every family present has at least **two** members standing in the line.

A family ID is just a label in the range 1 to n .

You are also given q constraints. Each constraint is a pair (x, y) , which means there must exist *at least* one member of family x that appears before *at least* one member of family y in the line.

In one operation, one person steps out of the line and then steps back into the line at any position (at the beginning, at the end, or between two adjacent people).

Find the minimum number of operations required to satisfy all constraints. It can be proven that it is always possible to satisfy all constraints with a finite number of moves.

Input Format

- The first line of input will contain a single integer t , denoting the number of test cases.
- Each test case consists of multiple lines of input.
 - The first line of each test case contains two space-separated integers n and q .
 - The second line contains n space-separated integers $a_1, a_2, \dots, a_n$.
 - The i -th of the next q lines contains two integers x_i and y_i , denoting the i -th constraint.

Output Format

For each test case, print one integer — the minimum number of required operations.

Constraints

- $1 \leq t \leq 100$
- $4 \leq n \leq 500$
- $1 \leq q \leq 500$
- $1 \leq a_i \leq n$
- $1 \leq x_i, y_i \leq n$ and $x_i \neq y_i$
- Both x_i and y_i appear at least once in a .
- All ordered pairs (x_i, y_i) are distinct.
- Every family with a positive number of people present has at least two members in the line.
- Each test case contains at least two distinct family IDs.
- The sum of n and the sum of q over all test cases each won't exceed 500.

Sample Input 1

```
3
4 2
2 2 1 1
1 2
2 1
6 3
1 2 1 2 3 3
```

```

1 2
2 3
3 1
6 2
2 2 1 1 3 3
1 2
3 1

```

Sample Output 1

```

1
1
2

```

Explanation

- **Test case 1:** Start with $[2, 2, 1, 1]$. Move one person from family 1 to the front and get $[1, 2, 2, 1]$. Now both constraints $1 \rightarrow 2$ and $2 \rightarrow 1$ hold, so the answer is 1.
- **Test case 2:** The initial array is $[1, 2, 1, 2, 3, 3]$ and the constraints are: $1 \rightarrow 2$, $2 \rightarrow 3$, and $3 \rightarrow 1$. Only $3 \rightarrow 1$ is false initially. Move one person from family 3 to the front, to obtain $[3, 1, 2, 1, 2, 3]$. Now all constraints are satisfied, so the answer is 1.
- **Test case 3:** The initial array is $[2, 2, 1, 1, 3, 3]$ and the constraints are: $1 \rightarrow 2$ and $3 \rightarrow 1$. Both are false initially. One valid way is:
 - move one person from family 3 to the front: $[3, 2, 2, 1, 1, 3]$;
 - move one person from family 1 to the second position: $[3, 1, 2, 2, 1, 3]$.

So the answer is at most 2. One move is not enough:

- if we move a person from family 3, then $1 \rightarrow 2$ is still false;
- if we move a person from family 2, then $3 \rightarrow 1$ is still false;
- if we move a person from family 1, we can fix at most one of the two constraints in one move.

Hence the minimum is 2.

C - Bitwise Operation Equations

There are n hidden operation variables $op_1, op_2, \dots, op_n$. Each of them is one of three bitwise operators: $\&$, $|$, or $\oplus$, representing bitwise AND, OR, and XOR, respectively.

You are also given m equations. The i -th equation is:

$$a_i \ op_{l_i} \ b_i = c_i \ op_{r_i} \ d_i$$

where all values a_i, b_i, c_i, d_i are bits (each is either 0 or 1).

Determine whether it is possible to assign an operator to every op_i so that all m equations are satisfied.

If there are multiple valid assignments, output any of them.

Input Format

- The first line of input will contain a single integer t , denoting the number of test cases.
- Each test case consists of multiple lines of input.
 - The first line of each test case contains two space-separated integers n and m .
 - The i -th of the next m lines contains six integers $a_i, b_i, c_i, d_i, l_i, r_i$.

Output Format

For each test case, if there is no valid assignment, output NO.

Otherwise, output YES on one line and then output one string of length n on the second line. The i -th character of this string must be one of $\&$, $|$, or $\wedge$, and it denotes the chosen value of op_i (bitwise AND, OR, XOR respectively.)

If there are multiple valid assignments, any of them will be accepted.

Constraints

- $1 \leq t \leq 10^4$
- $1 \leq n, m \leq 2 \cdot 10^5$
- $a_i, b_i, c_i, d_i \in \{0, 1\}$
- $1 \leq l_i, r_i \leq n$
- The sum of n and the sum of m over all test cases each won't exceed $2 \cdot 10^5$.

Sample Input 1

```
3
3 2
0 0 1 0 1 2
1 1 0 1 3 2
2 2
0 0 1 1 1 2
0 0 1 0 1 2
2 2
1 0 0 0 1 1
1 1 1 0 1 2
```

Sample Output 1

YES
|&^
NO
YES
&^

Explanation

- **Test case 1:** One valid assignment is |&^.
- **Test case 2:** The constraints force the same variable to be both & and ^, so no assignment exists.

D - Degree-Constrained MST

You are given a simple connected undirected graph with n vertices and m edges.

For each $i = 1, 2, \dots, m$, the i -th edge in the input connects vertices u_i and v_i , and has a weight of 2^i .

You are also given an integer k ($\lceil \frac{n}{2} \rceil \leq k \leq n$).

Compute the minimum possible total weight of a spanning tree of the graph, such that no vertex in the spanning tree has a degree greater than k .

The degree of a vertex x is the number of vertices y such that the edge (x, y) exists in the spanning tree.

If no such spanning tree exists, output -1 .

The answer may be large, so report it modulo 998 244 353.

Input Format

- The first line of input will contain a single integer t , denoting the number of test cases.
- Each test case consists of multiple lines of input.
 - The first line of each test case contains three space-separated integers n , m , and k .
 - The next m lines describe the edges. The i -th of these m lines contains two space-separated integers u_i and v_i , denoting an edge between u_i and v_i with a weight of 2^i .

Output Format

For each test case, output a single integer — the minimum possible weight modulo 998 244 353, or -1 if no valid spanning tree exists.

Constraints

- $1 \leq t \leq 1000$
- $2 \leq n \leq 2000$
- $n - 1 \leq m \leq \min\left(2000, \frac{n \cdot (n-1)}{2}\right)$
- $\lceil \frac{n}{2} \rceil \leq k \leq n$
- $1 \leq u_i, v_i \leq n$
- The graph is connected and has no self-loops or multiple edges.
- The sum of m over all test cases won't exceed 2000.

Sample Input 1

```
3
2 1 1
1 2
4 5 2
2 4
1 2
2 3
3 4
1 3
4 3 2
3 2
1 2
2 4
```

Sample Output 1

2
22
-1

Explanation

- **Test case 1:** There's only one edge with a weight of 2; and choosing it gives a spanning tree of the graph where each vertex has degree 1, which is valid because $k = 1$. So, the answer is 2.
- **Test case 2:** It's optimal to choose the first, second, and fourth edges to form a spanning tree of weight $2 + 4 + 16 = 22$. Note that while choosing the first, second, and third edges will result in a spanning tree with lower weight, it's not valid to do so because vertex 2 would have a degree of 3, exceeding $k = 2$.
- **Test case 3:** Only one spanning tree exists, and it is not valid.

E - Good Permutation

Suppose you have an array b of positive integers. The array b is called *good* if, for any two contiguous subarrays x and y of b , whenever $|x| > |y|$, it holds that $\text{sum}(x) > \text{sum}(y)$.

You are given an integer n . Find a permutation[†] p of length n such that p is good. If no such permutation exists, print -1 .

[†]A permutation of length n is an array of length n that contains every integer from 1 to n exactly once each.

Input Format

- The first line of input will contain a single integer t , denoting the number of test cases.
- Each test case consists of one line of input, containing a single integer n .

Output Format

For each test case:

- If there is no good permutation of length n , print -1 .
- Otherwise, print n integers $p_1, p_2, \dots, p_n$ — a permutation of numbers from 1 to n that is good.

If there are multiple answers, any of them will be accepted.

Constraints

- $1 \leq t \leq 500$
- $1 \leq n \leq 500$
- The sum of n over all test cases won't exceed 500.

Sample Input 1

```
2
1
3
```

Sample Output 1

```
1
2 3 1
```

F - Intersecting Sets

You are given two positive integers n and m and an array k of length $n - 1$, where each k_i is positive.

You must choose n sets $S_1, S_2, \dots, S_n$, where S_i should be made as follows:

- Choose integers l_i and r_i such that $1 \leq l_i \leq r_i \leq m$.
- Define $S_i = \{l_i, l_i + 1, \dots, r_i\}$.

For each i ($1 \leq i < n$), there should be *at least* k_i elements in the intersection of S_i and S_{i+1} . More formally,

$$|S_i \cap S_{i+1}| \geq k_i \quad \text{for all } 1 \leq i < n$$

Your task is to count the number of ways to choose n sets as described above. Since the number can be large, compute it modulo 998 244 353.

Two ways are considered **different** if there exists at least one index i such that the corresponding sets S_i are not equal.

Input Format

- The first line of input will contain a single integer t , denoting the number of test cases.
- Each test case consists of two lines of input.
 - The first line of each test case contains two space-separated integers n and m .
 - The second line contains $n - 1$ space-separated integers $k_1, \dots, k_{n-1}$.

Output Format

For each test case, output a single integer denoting the number of ways modulo 998 244 353.

Constraints

- $1 \leq t \leq 1000$
- $2 \leq n \leq 2500$
- $1 \leq m \leq 2500$
- $1 \leq k_i \leq m$
- The sum of n and the sum of m over all test cases each won't exceed 2500.

Sample Input 1

```
6
2 2
2
3 2
2 1
5 3
1 3 1 1
4 4
4 4 1
6 100
1 2 3 1 1
3 2
1 1
```

Sample Output 1

1
3
156
10
241576097
17

Explanation

- **Test case 1:** The only way to choose is $S_1 = \{1, 2\}$, $S_2 = \{1, 2\}$.
- **Test case 2:** The ways to choose S_1, S_2, S_3 are as follows:
 - $S_1 = \{1, 2\}$, $S_2 = \{1, 2\}$, $S_3 = \{1\}$.
 - $S_1 = \{1, 2\}$, $S_2 = \{1, 2\}$, $S_3 = \{2\}$.
 - $S_1 = \{1, 2\}$, $S_2 = \{1, 2\}$, $S_3 = \{1, 2\}$.

Note that $S_1 = \{1, 2\}$, $S_2 = \{2\}$, $S_3 = \{1, 2\}$ is not valid since $|S_1 \cap S_2| = 1k_1$.

G - Maximum Frequency

You are given an array a of length n , where $0 \leq a_i \leq n$ for all $1 \leq i \leq n$.

You must apply the following operation **exactly once**:

- Choose a subsequence[†] of indices $i_1 < i_2 < \dots < i_k$. The subsequence may be empty ($k = 0$).
- Let m be the MEX of the multiset $\{a_{i_1}, a_{i_2}, \dots, a_{i_k}\}$.
- For every chosen index i_j , replace a_{i_j} with $a_{i_j} \oplus m$, where $\oplus$ is bitwise XOR.

The score of the resulting array is the maximum frequency of any value in it.

Formally, if the resulting array is still denoted by a , then

$$\text{score}(a) = \max_x |\{i \mid a_i = x\}|$$

Find the maximum possible score.

[†]A sequence x is a subsequence of a sequence s if x can be obtained from s by the deletion of several (possibly, zero or all) elements from arbitrary positions.

Input Format

- The first line of input will contain a single integer t , denoting the number of test cases.
- Each test case consists of two lines of input.
 - The first line of each test case contains a single integer n .
 - The second line contains n space-separated integers $a_1, a_2, \dots, a_n$.

Output Format

For each test case, output one integer — the maximum score that can be obtained.

Constraints

- $1 \leq t \leq 10^4$
- $1 \leq n \leq 5 \cdot 10^5$
- $0 \leq a_i \leq n$
- The sum of n over all test cases won't exceed $5 \cdot 10^5$.

Sample Input 1

```
3
5
0 1 1 2 4
3
1 0 0
3
3 2 0
```

Sample Output 1

```
3
3
2
```

Explanation

- **Test case 1:** One optimal move is to choose only index 1. Then the chosen multiset is $\{0\}$, so $m = 1$. After XOR, the array becomes $[1, 1, 1, 2, 4]$, and the best frequency is 3.

H - Optimal Splitting

You are given a string s of length n consisting of lowercase Latin letters.

You need to partition s into two disjoint subsequences[†] x and y such that each character of s belongs to exactly one of the two subsequences.

Without loss of generality, assume $|x| \geq |y|$ (otherwise, swap x and y). Append character **a** to the end of y until $|y| = |x|$.

Now define a string z of length $|x|$ such that $z_i = \max(x_i, y_i)$ for every $1 \leq i \leq |x|$.

Find the lexicographically smallest possible string z .

[†]A sequence x is a subsequence of a sequence s if x can be obtained from s by the deletion of several (possibly, zero or all) elements from arbitrary positions.

Input Format

- The first line of input will contain a single integer t , denoting the number of test cases.
- Each test case consists of two lines of input.
 - The first line of each test case contains a single integer n .
 - The second line contains a string s of length n .

Output Format

For each test case, output one line containing the lexicographically smallest possible string z .

Constraints

- $1 \leq t \leq 10^4$
- $1 \leq n \leq 400$
- s is a string of length n and contains only lowercase Latin letters.
- The sum of n^3 over all test cases won't exceed 400^3 .

Sample Input 1

```
3
1
a
2
za
5
ababa
```

Sample Output 1

```
a
z
aba
```

Explanation

- **Test case 3:** Choose $x = \mathbf{aba}$ and $y = \mathbf{ab}$. After appending one **a** to y , we get $y = \mathbf{aba}$. z equals the pointwise maximum of x and y , which gives $z = \mathbf{aba}$. Hence, the answer is **aba**.

I - Restricted Removals

You are given an integer array a of length n .

Alice chooses a **binary** string b of length n with exactly k ones.

If $b_i = 1$, index i is protected. If $b_i = 0$, Bob may choose index i in his moves.

The string b is fixed for the whole game. The indices in b always refer to original indices $1, 2, \dots, n$. The string b itself never changes.

Then Bob makes zero or more moves. Bob's score initially equals 0. In one move, Bob chooses an index i such that:

- $b_i = 0$;
- $1 \leq i \leq |a|$.

Bob takes the element currently at index i , adds its value to his score, and then removes it from a . After that, the array shrinks by one.

If Bob chooses indices $p_1, p_2, \dots, p_m$ across moves, they must be non-increasing: $p_1 \geq p_2 \geq \dots \geq p_m$. Bob may choose the same index multiple times. Alice wants to minimize Bob's score, and Bob wants to maximize it.

Find Bob's final score if both players play optimally.

Input Format

- The first line of input will contain a single integer t , denoting the number of test cases.
- Each test case consists of two lines of input.
 - The first line of each test case contains two space-separated integers n and k .
 - The second line contains n space-separated integers $a_1, a_2, \dots, a_n$.

Output Format

For each test case, print one integer — Bob's final score.

Constraints

- $1 \leq t \leq 10^5$
- $1 \leq n \leq 5000$
- $0 \leq k \leq n$
- $-10^9 \leq a_i \leq 10^9$
- The sum of n^2 over all test cases won't exceed 5000^2 .

Sample Input 1

```
6
5 2
4 -1 7 -3 2
4 0
-5 -2 -1 -3
4 4
2 -7 5 1
1 0
-1
2 1
```

```
0 1
3 1
1 -1 1
```

Sample Output 1

```
7
0
0
0
1
1
```

Explanation

- **Test case 1:** Alice can choose $b = 10001$. Then, initially, the indices available to Bob are 2, 3 and 4 with values $-1, 7$ and -3 . Bob can guarantee a score of 7 by taking index 3 once (adding value 7), and any further legal move adds a non-positive value. Alice can force that Bob cannot do better than 7, so the answer is 7.
- **Test case 2:** As $k = 0$, Alice has no choice, $b = 0000$. All indices are available to Bob, but every array value is negative. Any move would decrease Bob's score, so Bob's optimal play is to make no move. Therefore, the answer is 0.
- **Test case 3:** As $k = n$, Alice has no choice, $b = 1111$. Every position is protected, so Bob has no legal move at all. Hence, Bob's score is 0.

J - Score Queries

Let b be an array of length m . The score of b is the number of indices i ($2 \leq i \leq m - 1$) such that there exist indices x and y satisfying $1 \leq x < i < y \leq m$ and

$$2 \cdot b_i > b_x + b_y$$

You are given an array a of length n .

You need to answer q queries on the array.

For each query, you are given two integers l and r ($1 \leq l \leq r \leq n$), and you must find the score of the subarray $[a_l, a_{l+1}, \dots, a_r]$.

Input Format

- The first line of input will contain a single integer t , denoting the number of test cases.
- Each test case consists of multiple lines of input.
 - The first line of each test case contains two space-separated integers n and q .
 - The second line contains n space-separated integers $a_1, a_2, \dots, a_n$.
 - Each of the next q lines contains two space-separated integers l and r , describing a query.

Output Format

For each query in each test case, output one integer — the score of the subarray $[a_l, a_{l+1}, \dots, a_r]$.

Constraints

- $1 \leq t \leq 10^4$
- $3 \leq n \leq 5 \cdot 10^5$
- $1 \leq q \leq 5 \cdot 10^5$
- $1 \leq a_i \leq n$
- $1 \leq l \leq r \leq n$
- $r - l + 1 \geq 3$
- The sum of n and the sum of q over all test cases each won't exceed $5 \cdot 10^5$.

Sample Input 1

```
3
5 4
2 5 1 3 4
1 5
2 4
1 3
3 5
3 1
1 2 3
1 3
4 1
1 3 2 4
1 4
```

Sample Output 1

2
0
1
1
0
1

Explanation

- **Test case 1:** For query $[1, 5]$ we have $a = [2, 5, 1, 3, 4]$ and valid middle positions are $i = 2$ and $i = 4$, so the score is 2.

K - Sorting Is Fun

For any array b , define $\text{score}(b)$ as follows.

Start with a variable $x = 0$. In one operation, choose a non-empty subsequence[†] s of the current array b . Then:

- update $x \leftarrow \max(x, \max(s) - \min(s))$;
- sort the elements of s in non-decreasing order;
- write the sorted elements back to the same chosen positions of b .

You may apply operations any number of times.

The value $\text{score}(b)$ is the minimum possible final value of x such that, after some sequence of operations, the whole array b becomes non-decreasing.

You are given an array a of length n .

You are also given m updates. In each update, two integers p and y are given, and you must set $a_p = y$. The updates are **persistent**.

After each update, output $\text{score}(a)$ for the current array.

[†]A sequence s is a subsequence of a sequence b if s can be obtained from b by the deletion of several (possibly, zero or all) elements from arbitrary positions.

Input Format

- The first line of input will contain a single integer t , denoting the number of test cases.
- Each test case consists of multiple lines of input.
 - The first line of each test case contains two space-separated integers n and m .
 - The second line contains n space-separated integers $a_1, \dots, a_n$.
 - Each of the next m lines of each test case contains two integers p and y ($1 \leq p \leq n$, $1 \leq y \leq n$), meaning that you must set $a_p = y$.

Output Format

For each test case, after each update, print one integer on a separate line — the value of $\text{score}(a)$ after applying this update.

Constraints

- $1 \leq t \leq 10^4$
- $1 \leq n, m \leq 5 \cdot 10^5$
- $1 \leq a_i, p, y \leq n$
- The sum of n and the sum of m over all test cases each won't exceed $5 \cdot 10^5$.

Sample Input 1

```
2
4 4
4 4 1 3
4 4
3 4
```

```
3 2
2 3
6 1
2 1 4 4 4 5
3 5
```

Sample Output 1

```
3
0
2
1
1
```

Explanation

- **Test case 1:** After the first update, the array becomes $[4, 4, 1, 4]$, and the answer is 3. After the second update, the array becomes $[4, 4, 4, 4]$, which is already sorted, so the answer is 0.

L - Sum K

You are given two integers n and k .

Your task is to find any permutation[†] p of the integers $\{1, 2, \dots, n\}$ such that

$$\sum_{i=1}^{n-1} \gcd(p_i, p_{i+1}) = k$$

That is, the sum of the GCD-s of adjacent elements of p must equal k .

If many such permutations exist, you may find any of them. If no such permutations exist, print -1 instead.

It is guaranteed that every test other than the sample has $n = 10^6$.

[†]A permutation of length n is an array of length n that contains every integer from 1 to n exactly once each.

Input Format

The only line of input will contain two space-separated integers n and k .

Output Format

If no valid permutation exists, output the single integer -1 .

Otherwise, output n space-separated integers $p_1, p_2, \dots, p_n$ denoting the permutation you found. If there are multiple valid permutations, any of them will be accepted.

Constraints

- $n = 8$ or $n = 10^6$
- $1 \leq k \leq 10^9$
- **Every test other than the sample has $n = 10^6$.**

Sample Input 1

8 10

Sample Output 1

3 2 5 4 8 7 1 6

Explanation

- **Test case 1:** It can be verified that the sum of adjacent GCD's of the given permutation equals 10.